Distributed Tsetlin Machines

1st Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

2nd Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

***Abstract*—Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.**

***Index Terms*—Cellular Automata, Learning Automata, Tsetlin Machines**

I. Introduction and Motivation

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.

II. Background

A. Cellular Automata - CA

To understand cellular automata, we first define the concept of an Automaton.

1) Automata: Automata is the plural form of **Automaton**, which is an abstract computational model characterized by a finite internal state. The state of an automaton evolves over time according to a state transition function.

At each discrete time step, the next state depends on the previous state and the current external input.

The state transition function can be written as:

$$s_{t+1} = f(s_t, x_t) \tag{1}$$

where s_t represents the current state, x_t represents the input at time step t, and the set of possible states is discrete and finite [1].

For example, a flip-flop circuit can be modeled as a simple automaton with a binary state set $\{0, 1\}$.

2) CA Spatial Grid: A Cellular Automata is a network of Automaton, that are arranged in a regular spatial grid. The grid can theoretically be defined in N dimensions, but one- and two-dimensional grids are the most commonly studied cases. Additionally, both space (the grid) and time are discrete. Time progresses in uniform steps, and at each step the states of cells are updated based on a uniform state transition function that is applied to all cells in the grid [1].

3) Boundaries: The cells at the edge of the grid do not have a complete neighborhood, and hence would have a different behavior compared to the rest of the cells, breaking the uniformity of a CA. Hence, it is important to define boundaries as one of the following types: **No boundaries**: this assumes that the grid is surrounded by vacuum cells that that are filled by "0" or blank. Those cells are fixed and do not change. **Periodic boundaries**: this assumes that the grid is wrapped around. The edges of each dimension/axis are connected to each other. The grid takes the shape of a ring in a 1-D CA, and a doughnut in 2-D CA.

Cut-off boundaries: this assumes that the cells at the edges do not have neighbors beyond the border of the grid. This requires the state transition function to be modified to include such cases. **Fixed boundaries**: this assumes that the no cells are beyond the borders, and that cells on the boundaries have fixed states [1].

4) Neighborhood: The neighborhood of a cell is defined as a set of relative indices with respect to the cell's position. A neighborhood with radius r includes all cells within distance r from the current cell.

For a one-dimensional grid, the neighborhood is defined as:

$$\mathcal{N} = \{-r, \ldots, -1, 0, 1, \ldots, r\}.$$

For a two-dimensional grid, the neighborhood of a cell (i, j) is defined as:

$$\mathcal{N}(i, j) = \{(i + dx, j + dy) \mid dx, dy \in \{-r, \ldots, r\}\} \tag{2}$$

For $r = 1$, the Von Neumann neighborhood considers only the cardinal directions, defined by the relative index set:

$$\mathcal{N}_{VN} = \{(0, 1), (0, -1), (1, 0), (-1, 0)\} \tag{3}$$

The Moore neighborhood extends this to include diagonal neighbors:

$$\mathcal{N}_M = \{(\pm 1, 0), (0, \pm 1), (\pm 1, \pm 1)\} \tag{4}$$

5) State Set: The State set, denoted by ϕ, is a finite set of discrete states which a cell can take.

$$\phi = \{0, 1, 2, \ldots, k - 1\} \tag{5}$$

6) State Transition Function: Each cell in the grid has an automaton with a state transition function (as in equation (1)) that updates the state such that the next state of the current cell (s_{t+1}) depends on its current state (s_t) and the states of the cells in its neighborhood (x_t). A key property of Cellular Automata is uniformity: all cells share the same state transition function and neighborhood structure.

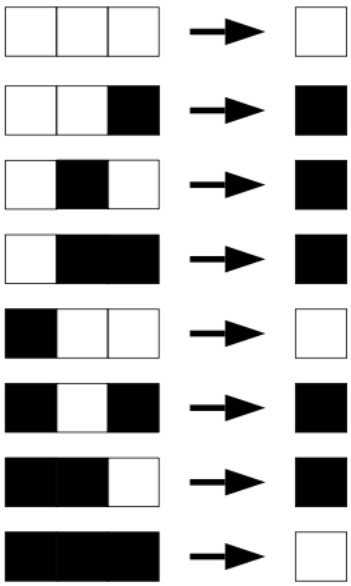

Fig. 1. Rule 110

A more precise representation for the transition function for a 1-D CA is defined as:

$$s_i^{t+1} = f(s_j^t \mid j \in \mathcal{N}(i)) \tag{6}$$

where:

- s_i^{t+1} is the next state of cell i
- s_j^t is the current state of neighboring cell j at time t
- $\mathcal{N}(i)$ is the neighborhood of cell i

For a 2-D CA, the transition function extends to two dimensions:

$$s_{i,j}^{t+1} = f(s_{i+dx,\ j+dy}^t \mid (dx, dy) \in \mathcal{N}) \tag{7}$$

where:

- $s_{i,j}^{t+1}$ is the next state of the cell at position (i, j)
- $s_{i+dx,\ j+dy}^t$ is the current state of a neighboring cell at relative offset (dx, dy)
- $\mathcal{N}$ is the neighborhood index set as defined in equation (2)

The transition function f can be defined in several ways. It can be specified as a fixed rule, such as the majority rule; where the next state is defined by the majority of current states in the neighborhood. Another famous rule is in Conway's Game of Life, where the next state of a cell is determined by counting the number of live neighbors. Specifically, a live cell survives if it has two or three live neighbors, dies otherwise, and a dead cell becomes alive if it has exactly three live neighbors [2].

7) 2-D Cellular Automata: Alternatively, f can be defined as a lookup table that explicitly maps each possible combination of neighborhood states to a next state. In an elementary 1-D CA, each cell has two states $\{0, 1\}$, and has a neighborhood of 3 cells (itself plus one to the right and one

Fig. 2. 1D-CA.

to the left). This neighborhood of 3 cells can have $2^3 = 8$ configurations. For each of those configurations, the next state of the cell can be either 0 or 1. So the total number of possible rules is $2^8 = 256$. Figure 1 shows one possible rule (Rule 110). Wolfram assigned a number for each possible Rule by interpreting the output of each lookup table as a binary number. so in figure 1 the lookup table can be interpreted as binary 01101110 (where top digit in the lookup table is on the far left, and each following digit is ordered correspondingly to its left) which translates to the decimal number 110. Note that the term rule refers to the whole lookup table (set of all possible configurations of the neighborhood and a certain corresponding update states), not to a single configuration of the neighborhood.

This scales up significantly in a 2-D CA: with a Von Neumann neighborhood of 5 cells, there are $2^5 = 32$ possible configurations, yielding 2^{32} possible rules; and with a Moore neighborhood of 9 cells, there are $2^9 = 512$ possible configurations, yielding 2^{512} possible rules.

More generally, f can be expressed as a mathematical function or a higher-level algorithm (which can be as complex as Neural Networks) [2].

8) Formal Definition: A d-dimensional CA is defined by the tuple $\langle \mathbb{Z}^d, \phi, \mathcal{N}, f \rangle$, where:

- $\mathbb{Z}^d$ is a d-dimensional lattice of integer d-tuples representing the cell positions
- ϕ is the finite set of possible cell states
- $\mathcal{N}$ is the neighborhood index set
- f is the state transition function

[3]

B. Types of CA

According to the formal definition we formulated above, we can have different types of CA by modifying one or more of the tuple elements $(\mathbb{Z}^d, \phi, \mathcal{N}, f)$. The simplest kind of CA is what is defined by Wolfram as the **Elementary CA**; a CA with one dimension ($Z^d =$), two states ($\phi =$) and a neighborhood of adjacent cells ($\mathcal{N} =$) [4].

A **2-D CA** in its simplest form is defined exactly like the elementary CA but with a two-dimensional lattice ($\mathbb{Z}^d = \mathbb{Z}^2$), two states ($\phi = \{0, 1\}$), and a defined neighborhood $\mathcal{N}$.

Other types of Cellular Automata include **Higher-Dimensional CA**, which extend the grid to three or more spatial dimensions; **Multi-Layer CA**, where the cell state is extended from a scalar to a vector of variables (e.g. an RGB vector); and **Asynchronous CA**, where the synchronous updating assumption is relaxed to allow sequential cell updates. Another type worth defining more precisely is the Stochastic CA.

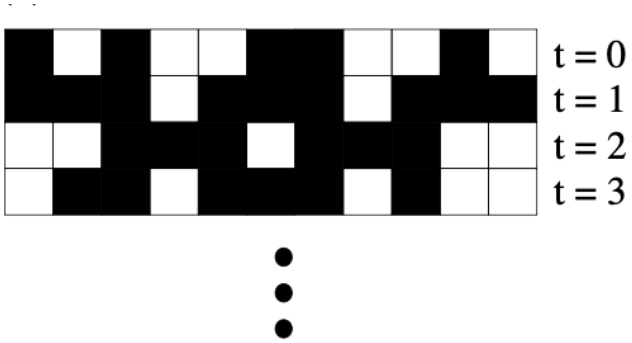

Fig. 3. space-time plot of an elementary CA.

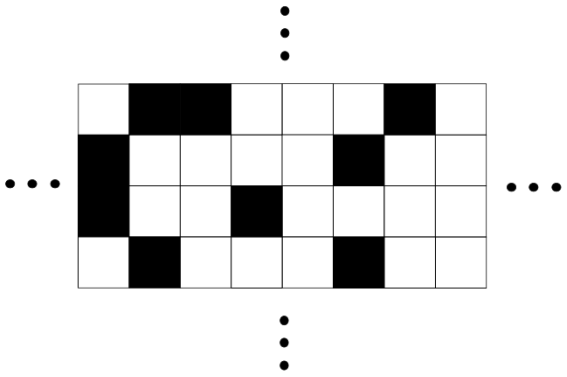

Fig. 4. 2d CA.

1) Stochastic Cellular Automata (SCA): Cellular Automata as defined so far are deterministic. Their state transition function produce the exact same output given the same inputs. A Stochastic Cellular Automaton extends the deterministic CA by introduces randomness using a probabilistic state transition function, where an additional random variable $r \sim \mathcal{U}(0,1)$ is introduced. The next state of cell i at time $t+1$ is therefore given by:

$$s_i^{t+1} = f(s_j^t \mid j \in \mathcal{N}(i),\ r), \quad r \sim \mathcal{U}(0,1) \tag{8}$$

For example, if a cell becomes active with probability 0.4 when at least one neighbor is active, a random value $r \in [0,1]$ is sampled and the next state is set to 1 if $r < 0.4$, and 0 otherwise.

C. Space-Time Diagram

In order to understand how simple local rules can produce complex global behavior, Wolfram studied CA in its simplest form — the Elementary CA — using space-time diagrams.

Figure 3 shows a space-time diagram for an Elementary CA. The top row is the one-dimensional lattice with periodic boundaries and a random initial configuration at time $t = 0$. Each successive row shows the updated states of the lattice at the next time step [4]. These diagrams are useful for tracking the spatial configuration of a CA over a number of time steps. Wolfram conducted an exhaustive survey of all 256 Elementary CA rules using space-time diagrams, testing each rule against many different initial configurations, and accordingly classified the observed patterns into 4 classes.

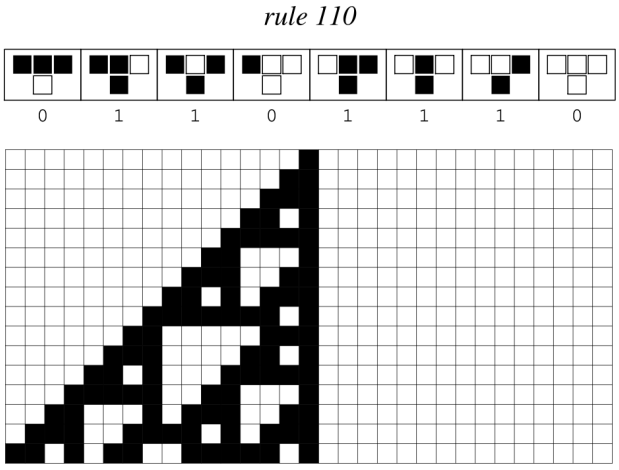

Fig. 5. Space-time diagram of Rule 110.

D. The 4 Classes

1) Class 1: Almost any initial configuration of the lattice converges to the same uniform final pattern after a number of time steps. The most obvious example is Rule 0 as it maps every neighborhood configuration to a dead cell, causing the entire lattice to go dark regardless of the initial configuration.

2) Class 2: Almost any initial configuration of the lattice converges to a uniform final pattern, or cycles between a small number of repeating patterns.

3) Class 3: Almost any initial configuration produces random-looking, chaotic behavior. Any small change in the initial conditions affects the entire evolution of the system. For example, Rule 30 belongs to this class, as even a single live cell in an otherwise dead lattice generates a highly irregular, unpredictable pattern that never stabilizes.

4) Class 4: Almost any initial configuration produces localized, persistent structures that interact with each other in complex ways that cannot be predicted from the simple local rules alone. Most famous example is Rule 110.

E. Learning Automata - LA

A Learning Automaton is a theoretical machine which learns an optimal action out of a defined set of actions through repeated interactions with a random unknown environment [3].

An LA is very similar to the Automaton defined earlier, but with a key modification. In a basic Automaton, the next state is a function of the current state and an input x. In an LA, the current state is replaced by the last selected action, the input is replaced by the reinforcement feedback signal received from the environment based on that selected action, and the next state is replaced by the updated action selection structure.

At each step the the LA selects an action from its allowed action set. The Selection is based on the Automaton's learning of what is the optimal action. The action is applied to the random environment which in return responds with a feedback (reward or penalty). Based on the Feedback the LA updates updates its learning strategy for action selection. LA's can vary in terms of how the action set and the learning strategy are defined.

1) Action Set: LAs reported in the literature can be classified based on their action set into two classes: Finite Action-Set Learning Automata (FALA) and Continuous Action-Set Learning Automata (CALA).

FALA is an LA where the action set is defined as a discrete finite set of actions (e.g. move right or left). The majority of classical LAs fall under this class (e.g. Tsetlin, Krinsky, and Krylov Automata) [5]. In some cases, the LA interacts with the environment using a continuous-valued parameter (e.g. choosing a speed value), in which case the action set is defined as a continuous interval and the LA is classified as a CALA. However, CALA is less common in the literature [6].

2) Action Selection Strategy: LAs can further be classified into two classes based on their internal structure, which determines the action selection strategy: Fixed Structure Stochastic Automata (FSSA) and Variable Structure Stochastic Automata (VSSA).

FSSA: In an FSSA, the LA has a finite set of states arranged in a fixed graph. Transitions between states follow fixed rules, and the graph structure itself does not change. The current state of the LA determines the selected action, and transitions are triggered by the environment's feedback (reward or penalty) on the previous action — this is how learning is achieved [7].

Tsetlin Automaton: The first LA was introduced by M. L. Tsetlin, and is known as the Tsetlin Automaton. It falls under FALA, as it has a finite action set of only two actions, and is classified as FSSA due to the following fixed internal structure.

The automaton has $2N$ states, divided equally into two halves of N states each, where each half corresponds to one of the two actions. The current state determines which action is selected — states in the first half produce action a_1, and states in the second half produce action a_2. The depth of the state within its half reflects the automaton's confidence in that action; the deeper the state, the greater the confidence [8].

Upon receiving a reward, the automaton moves one step deeper into its current half, reinforcing the current action choice. Upon receiving a penalty, the automaton moves one step back in the direction of the other action. This fixed transition structure does not change during learning — only the automaton's current position (state) within it changes [8].

Several variants of this internal structure have been reported in the literature. Some modify the transition rules to produce a scaled transition, where the step size is determined by the magnitude of the feedback. The Krylov Automaton, for instance, follows the same reward transition as Tsetlin, but introduces stochasticity on penalty — the automaton either stays at its current state or moves back, each with some probability. More complex variants further modify the transition rules, alter the internal graph structure, or extend the automaton to support more than two actions.

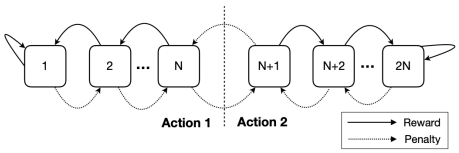

Fig. 6. Tsetlin Automaton

VSSA: Unlike FSSA, Variable Structure Stochastic Automata do not have a graph structure or discrete states. Instead, a VSSA maintains an action selection probability vector $\mathbf{p} = (p_1, p_2, \ldots, p_n)$, where each p_i is the probability of selecting action a_i. Learning is achieved by updating this probability vector based on the received feedback. The term "variable structure" reflects the fact that this probability distribution itself changes over time as the automaton learns [7].

A Learning Automaton is formally defined as a tuple:

$$LA = \langle A, F, P, T \rangle \tag{9}$$

where:

- $A = \{a_1, a_2, \ldots, a_r\}$ is the finite set of actions
- $F = \{\text{reward}, \text{penalty}\}$ is the feedback set from the environment, sometimes written as $\{0, 1\}$ or $\{\beta = 0, \beta = 1\}$
- P is the internal representation — either a finite state set (for FSSA) or a probability vector (for VSSA)
- T is the update rule — either a state transition function (for FSSA) or a probability update function (for VSSA)

The two classifications of LA based on internal structure, FSSA and VSSA, differ in how P and T are concretely defined.

In our research we are focusing on the study of Tsetlin Automata. Accordingly, a formal definition of it is a tuple $\langle A, F, P, T \rangle$, where each component is concretely instantiated as follows. The action set is $A = \{a_1, a_2\}$, and the feedback set is $F = \{\text{reward}, \text{penalty}\}$. The internal representation P is a finite set of $2N$ states $S = \{s_1, s_2, \ldots, s_{2N}\}$, partitioned into two halves of N states each, where states s_1 through s_N correspond to action a_1, and states s_{N+1} through s_{2N} correspond to action a_2. The current action is determined by an output function $G : S \to A$:

$$G(s_u) = \begin{cases} a_1, & \text{if } 1 \leq u \leq N \\ a_2, & \text{if } N+1 \leq u \leq 2N \end{cases} \tag{10}$$

The update rule $T : S \times F \to S$ is defined as:

$$T(s_u, \beta) = \begin{cases} s_{u+1}, & \text{if } 1 \leq u \leq N \text{ and } \beta = \text{penalty} \\ s_{u-1}, & \text{if } N+1 \leq u \leq 2N \text{ and } \beta = \text{penalty} \\ s_{u-1}, & \text{if } 1 < u \leq N \text{ and } \beta = \text{reward} \\ s_{u+1}, & \text{if } N+1 \leq u < 2N \text{ and } \beta = \text{reward} \\ s_u, & \text{otherwise} \end{cases} \tag{11}$$

F. Cellular Learning Automata - CLA

A Cellular Learning Automaton (CLA) is a system that combines the spatial structure of a Cellular Automaton with the learning capability of a Learning Automaton. Each cell in the grid is assigned a Learning Automaton, which can be of any kind depending on the use case. Note that what a standard CA calls the cell's *state* corresponds here to the LA's selected *action*; this distinction is important, as the LA's internal state and its selected action are separate concepts.

The LA is responsible for selecting an action at each time step. The environment for each LA is defined by its neighborhood: the neighboring cells' actions serve as input to a feedback function, which maps them to a reward or penalty signal for the current cell. This feedback function is designed based on the desired collective behavior. For example, if the goal is synchronization, a reward is given when all cells in the neighborhood select the same action. This feedback mechanism is strictly local, depending only on the neighborhood rather than the global state of the grid, which promotes the distributed nature of the CLA. Alternatively, feedback can be derived from a desired macroscopic behavior of the grid as a whole.

The overall flow of a CLA proceeds as follows. At each time step, every cell selects an action based on its LA's current internal state (for FSSA) or probability vector (for VSSA). Each cell then observes the actions of its neighbors, and a local feedback function maps those actions to a reward or penalty signal. Based on that signal, the LA updates its internal state or probability vector exactly as a standard LA would.

A Tsetlin-based Cellular Learning Automaton is formally defined by the tuple $\langle \mathbb{Z}^d, \mathcal{N}, \mathcal{T}, \rho \rangle$, where:

- $\mathbb{Z}^d$ is the d-dimensional spatial grid of cells
- $\mathcal{N}$ is the neighborhood index set
- $\mathcal{T} = \langle A, F, S, G, T \rangle$ is the Tsetlin Automaton assigned to each cell, where $A = \{a_1, a_2\}$, $F = \{\text{reward}, \text{penalty}\}$, $S = \{s_1, \ldots, s_{2N}\}$, and G, T are the output and transition functions as defined in equations (10) and (11)
- $\rho : A^{|\mathcal{N}|} \to F$ is the local feedback function, which maps the actions of neighboring cells to a reward or penalty signal for the current cell

III. Conclusion

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua.

References

[1] H. Sayama, *Introduction to the Modeling and Analysis of Complex Systems*. Albany: Open SUNY Textbooks, 2015.

[2] M. Mitchell, "Life and Evolution in Computers," *History and Philosophy of the Life Sciences*, vol. 23, no. 3/4, pp. 361–383, 2001. [Online]. Available: https://www.jstor.org/stable/23332520

[3] M. Ahangaran, N. Taghizadeh, and H. Beigy, "Associative cellular learning automata and its applications," *Applied Soft Computing*, vol. 53, pp. 1–18, Apr. 2017. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S156849461630624X

[4] M. Mitchell, *Complexity: A Guided Tour*. Oxford University Press, Apr. 2009.

[5] A. Yazidi, N. Bouhmala, and M. Goodwin, "A team of pursuit learning automata for solving deterministic optimization problems," *Applied Intelligence*, vol. 50, no. 9, pp. 2916–2931, Sep. 2020. [Online]. Available: https://doi.org/10.1007/s10489-020-01657-9

[6] W. Jiang, B. Li, S. Li, Y. Tang, and C. L. P. Chen, "A new prospective for Learning Automata: A machine learning approach," *Neurocomputing*, vol. 188, pp. 319–325, May 2016. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0925231215019086

[7] P. Aggarwal, C. Liu, and L. Levitin, "A New Class of Learning Automata: The Probabilistically-Switch-Action-on-Failure Automaton Part 1 – Analytical Models," Mar. 2022. [Online]. Available: https://www.techrxiv.org/doi/full/10.36227/techrxiv.19425761.v1

[8] O.-C. Granmo, "The Tsetlin Machine – A Game Theoretic Bandit Driven Approach to Optimal Pattern Recognition with Propositional Logic," Jan. 2021, arXiv:1804.01508 [cs]. [Online]. Available: http://arxiv.org/abs/1804.01508